Damietta University

Faculty of Computers & Artificial Intelligence

كلية الحاسبات والذكاء الاصطناعي · جامعة دمياط

MARKIO

Audio Watermarking via Multi-Objective Genetic Algorithm Optimizer

Soft Computing · University Curriculum Project · 2024–2025

Prepared by:

تامر السعيد شوقي الكوش

ID: 808869995 · Program: AI · Level 4

أحمد أسامة محمد المرسي

ID: 812584270 · Program: CS · Level 4

Mentored by:

Eng. Ibrahim Elgazzar

Supervised by:

Dr. Mona Elbedwehy

github.com/futurevoid/MARKIO

markio.streamlit.app

2024 – 2025

Table of Contents

1. Problem Statement

Digital audio watermarking embeds a hidden binary message inside an audio signal in a way that is imperceptible to listeners. The watermark must survive common distortions — noise, compression — so it can be recovered later for copyright verification or authentication.

Embedding strength controls a direct trade-off: stronger embedding improves robustness against noise attacks but degrades audio quality (lower SNR), while weaker embedding preserves quality but makes the watermark fragile. These two objectives cannot be simultaneously maximised.

A scalar weighted-sum approach collapses both objectives into a single value, which requires an arbitrary weighting decision and yields only one solution. MARKIO solves the problem as a multi-objective optimisation, producing a Pareto front — the complete set of solutions where neither objective can be improved without worsening the other.

Objectives (maximise both simultaneously):

SNR(α, band, n_carriers)	Signal-to-Noise Ratio in dB
BitAcc(α, band, n_carriers)	Fraction of bits recovered after noise attack (σ = 0.05)
Constraints	$\alpha \in [0.001, 0.15]$, band $\in [0.0, 1.0]$, n_carriers $\in \{1, 2, 3\}$

2. System Architecture

The GA only calls compute_objectives(). All signal processing is isolated below that interface.

generate_audio()	Synthesise a multi-tone test signal
generate_watermark()	Create a random 64-bit binary string
_make_carrier()	Build a shaped pseudo-random carrier for one bit segment
embed_watermark()	audio[s:e] += (alpha/nc) * symbol * carrier
extract_watermark()	Correlate received signal with carriers to recover bits
compute_snr()	10 * log10(P_signal / P_noise)
compute_objectives()	Returns (snr, acc) — never combined into a scalar
Individual	Chromosome: 3 genes + 3 self-adaptive step sizes
non_dominated_sort()	NSGA-II fast sort — Pareto ranks and fronts
crowding_distance()	Isolation measure along a Pareto front
GeneticAlgorithm	All operators and the main loop
plot_results()	12-panel matplotlib figure
run_ga() generator	Generator version for Streamlit — yields each generation

3. Audio & Watermark Generation

3.1 Audio Signal Synthesis

A deterministic sum of three harmonic sinusoids plus Gaussian background noise, normalised to [-1, 1]. A real audio file (drill.wav) is also included in the repository for use with soundfile.read().

```
def generate_audio(duration=2.0, sample_rate=8000, seed=42):
    rng = np.random.default_rng(seed)
    t   = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)
    sig = (0.5 * np.sin(2 * np.pi * 440  * t)
         + 0.3 * np.sin(2 * np.pi * 880  * t)
         + 0.1 * np.sin(2 * np.pi * 1320 * t)
         + 0.08 * rng.standard_normal(len(t)))
    return sig / np.max(np.abs(sig)), sample_rate
```

duration	2.0 s
sample_rate	8000 Hz
total samples	16 000
components	440 Hz · 880 Hz · 1320 Hz · Gaussian noise σ = 0.08

3.2 Binary Watermark

A fixed 64-bit binary string. The same sequence is used across all fitness evaluations.

```
def generate_watermark(length=64, seed=7):
    return np.random.default_rng(seed).integers(0, 2, size=length).astype(float)
```

4. Carrier Generation

Each watermark bit is embedded using a pseudo-random unit-norm carrier vector. Spread-spectrum watermarking distributes the watermark energy across all samples in a segment rather than concentrating it at specific frequencies.

band controls the carrier's spectral shape. band=1.0 produces a flat-spectrum white-noise carrier (maximum robustness). band=0.0 applies moving-average smoothing, producing a low-frequency carrier (lower perceptibility, lower robustness). The carrier seed is derived as i*4 + k, giving each carrier a unique deterministic sequence reproducible at extraction without transmission.

```
def _make_carrier(seg_len, band, carrier_seed):
    rng    = np.random.default_rng(carrier_seed)
    c      = rng.standard_normal(seg_len)
    window = max(1, int((1.0 - band) * seg_len * 0.15))
    if window > 1:
        c = np.convolve(c, np.ones(window) / window, mode='same')
    return c / (np.linalg.norm(c) + 1e-9)
```

5. Watermark Embedding & Extraction

5.1 Spread-Spectrum Embedding

The audio is split into N segments (one per bit). Each segment receives the carrier scaled by alpha/n_carriers, signed by the bit value. Dividing by n_carriers keeps total power constant across redundancy levels.

```
def embed_watermark(audio, watermark, alpha, band=1.0, n_carriers=1):
    nc      = max(1, int(round(n_carriers)))
    seg_len = len(audio) // len(watermark)
    out     = audio.copy()
    for i, bit in enumerate(watermark):
        s, e   = i * seg_len, (i + 1) * seg_len
        symbol = 1.0 if bit else -1.0
        for k in range(nc):
            c = _make_carrier(e - s, band, i * 4 + k)
            out[s:e] += (alpha / nc) * symbol * c
    return np.clip(out, -1.0, 1.0)
```

watermarked[s:e] = audio[s:e] + (α/nc) · symbol · carrier(i, k) for k ∈ {0, …, nc−1}

5.2 Watermark Extraction

The same carrier sequences are regenerated from the same seeds and correlated against the received signal. The sign of the summed correlations determines each decoded bit (correlation receiver / matched filter). Requires the original clean audio — informed (non-blind) detection model.

```
def extract_watermark(original, received, n_bits, band=1.0, n_carriers=1):
    nc      = max(1, int(round(n_carriers)))
    seg_len = len(original) // n_bits
    bits    = np.zeros(n_bits)
    for i in range(n_bits):
        s, e = i * seg_len, (i + 1) * seg_len
        diff = received[s:e] - original[s:e]
        corr = sum(np.dot(diff, _make_carrier(e-s, band, i*4+k))
                   for k in range(nc))
        bits[i] = 1.0 if corr >= 0 else 0.0
    return bits
```

6. Objective Metrics

6.1 Signal-to-Noise Ratio (SNR)

```
def compute_snr(original, watermarked):
    noise = watermarked - original
    pn    = np.mean(noise ** 2)
    return 10 * np.log10(np.mean(original**2) / pn) if pn > 1e-12 else 100.0

SNR = 10 * log10( E[x^2] / E[n^2] )   (dB)    threshold: > 25 dB
```

6.2 Bit Accuracy (Robustness)

Gaussian noise ($\sigma = 0.05$) is added to the watermarked signal, then bits are extracted and compared against the original. Range: 0.5 (random) to 1.0 (perfect).

```
attacked  = wm_audio + rng.normal(0, 0.05, len(wm_audio))
attacked  = np.clip(attacked, -1.0, 1.0)
extracted = extract_watermark(audio, attacked, len(watermark), ...)
acc       = float(np.mean(extracted == watermark))
```

6.3 compute_objectives()

Returns both objectives as a tuple. They are never combined.

```
def compute_objectives(alpha, band, n_carriers, audio, watermark):
    wm  = embed_watermark(audio, watermark, alpha, band, n_carriers)
    snr = compute_snr(audio, wm)
    attacked = np.clip(wm + noise, -1, 1)
    ext = extract_watermark(audio, attacked, ...)
    acc = float(np.mean(ext == watermark))
    return snr, acc
```

7. Individual — Chromosome Design

Each individual holds two arrays: genes (parameter values) and sigmas (per-gene mutation step sizes that evolve alongside the genes).

Chromosome

Gene	Name	Range	Type	Role
0	alpha (α)	[0.001, 0.15]	Continuous	Embedding amplitude scale
1	band	[0.0, 1.0]	Continuous	Carrier bandwidth (0=narrow, 1=broadband)
2	n_carriers	[1.0, 3.0]	Continuous → int	Redundant carriers per bit

Self-Adaptive Step Sizes

Initial sigmas	[0.010, 0.050, 0.200] for [alpha, band, n_carriers]
sigma_min	1e-5
Update rule	σ'_d = σ_d · exp(τ · N(0, 1))
τ (tau)	1 / sqrt(2 · n_genes) = 1 / sqrt(6) (Schwefel, 1981)

balanced_score() — used only by HoF, not selection

```
def balanced_score(self, snr_ref=40.0):
    return min(self.snr / snr_ref, 1.0) + self.acc
```

8. Pareto Utilities

8.1 Domination

A dominates B if A is at least as good in all objectives and strictly better in at least one.

```
def _dominates(a, b):
    return (a.snr >= b.snr and a.acc >= b.acc)
       and (a.snr >  b.snr or  a.acc >  b.acc)
```

8.2 Non-Dominated Sorting

Population sorted into fronts. Front 0 holds all non-dominated individuals. O(N²) — NSGA-II fast sort (Deb et al., 2002).

```
For each i:
    dom_count[i] = |{j : j dominates i}|
    dominated[i] = {j : i dominates j}
Front 0 <- {i : dom_count[i] == 0}
For each i in Front k:
    for j in dominated[i]:
        dom_count[j] -= 1
        if dom_count[j] == 0: Front k+1 <- j
```

8.3 Crowding Distance

Boundary individuals on a front receive distance ∞. Interior individuals receive the sum of normalised neighbour gaps across all objectives. Higher distance = preferred in selection.

```
for each objective f:
    sort front by f
    dist[boundary] = inf
    dist[m] += (f[m+1] - f[m-1]) / (f_max - f_min)
```

9. Genetic Algorithm

9.1 Initialization

Genes sampled uniformly within bounds. All individuals evaluated before generation 1.

```
genes = np.array([rng.uniform(*b) for b in Individual.BOUNDS])
pop.append(Individual(genes))
```

9.2 Pareto-Aware Tournament Selection

k individuals drawn at random. Winner: lowest Pareto rank; ties broken by highest crowding distance.

```
for i in candidates:
    if ranks[i] < ranks[best]:
        best = i
    elif ranks[i] == ranks[best] and crowd[i] > crowd[best]:
        best = i
```

9.3 Simulated Binary Crossover (SBX)

Polynomial distribution around parents, controlled by distribution index η. Applied per gene independently. Step sizes crossed arithmetically. (Deb & Agrawal, 1995)

```
u    = rng.random()
beta = (2*u)**(1/(eta+1))  if u <= 0.5  else  (1/(2*(1-u)))**(1/(eta+1))
child1 = clip(0.5 * ((1+beta)*p1.gene + (1-beta)*p2.gene))
child2 = clip(0.5 * ((1-beta)*p1.gene + (1+beta)*p2.gene))
sigma_child = 0.5 * (p1.sigma_d + p2.sigma_d)
```

9.4 Self-Adaptive Mutation

Step size updated first, then gene perturbed. Applied per gene independently.

```
sigma_d = max(sigma_d * exp(tau * N(0,1)),  sigma_min)
gene_d  = clip(gene_d + sigma_d * N(0,1),  lo, hi)
```

9.5 Stagnation Detection & Diversity Injection

If balanced_score of the HoF does not improve for patience generations, the injection_frac worst-ranked individuals are replaced with fresh randoms.

```
if no_improve >= patience:
    n_inj = int(injection_frac * pop_size)
    worst = sorted(range(N), key=lambda i: -ranks[i])[:n_inj]
    for i in worst:
        pop[i] = Individual(rng.uniform(*b) for b in BOUNDS)
```

9.6 Adaptive Tournament Pressure

k grows linearly from k_min at generation 0 to k_max at the final generation.

```
progress = gen / max(G - 1, 1)
k        = max(2, round(k_min + progress * (k_max - k_min)))
```

9.7 Hall of Fame (HoF)

Best-balanced individual seen across all generations. Inserted at position 0 of each new generation (elitism).

9.8 Main Loop

```
for gen in range(G):
    ranks, fronts = non_dominated_sort(pop)
    crowd         = all_crowding(pop, fronts)
    for ind in pop:
        if ind.balanced_score() > hof.balanced_score(): hof = ind.clone()
    # stagnation check + injection if triggered
    k       = _k(gen)
    new_pop = [hof.clone()]
    while len(new_pop) < pop_size:
        p1, p2 = _tournament(...), _tournament(...)
        c1, c2 = _sbx(p1, p2)
        c1, c2 = _mutate(c1), _mutate(c2)
        c1.evaluate(audio, watermark); c2.evaluate(audio, watermark)
        new_pop.extend([c1, c2])
    pop = new_pop[:pop_size]
ranks, fronts = non_dominated_sort(pop)
pareto_front  = [pop[i] for i in fronts[0]]
```

10. GA Feature Summary

#	Feature	Mechanism	Rationale
1	Self-Adaptive Mutation	sigma' = sigma * exp(tau * N(0,1)) per gene	Step sizes co-evolve; tau=1/sqrt(2*n_genes) (Schwefel, 1981)
2	Simulated Binary Crossover	Polynomial distribution, index eta	Standard crossover for real-coded GAs (Deb & Agrawal, 1995)
3	Non-Dominated Sorting	Pareto rank replaces scalar fitness	Correct MO treatment; no weight tuning (Deb et al., 2002)
4	Crowding Distance	Normalised neighbour gaps on front	Maintains solution spread along the Pareto front
5	Adaptive Tournament Pressure	k: k_min to k_max linearly	Shifts exploration to exploitation without manual scheduling
6	Stagnation + Injection	Replace bottom inj_frac on stagnation	Recovers diversity without discarding the HoF
7	Hall of Fame	Tracks best-balanced individual globally	Immune to regression after injection events

11. Hyperparameter Reference

Parameter	Default	Description
pop_size	40	Population size
generations	80	Number of iterations
sbx_eta	5	SBX distribution index
crossover_prob	0.85	Per-gene crossover probability
tournament_k_min	2	Tournament size at generation 0
tournament_k_max	6	Tournament size at final generation
stagnation_patience	12	Gens without HoF improvement before injection
injection_frac	0.30	Fraction replaced on stagnation
sigma_init	[0.010,0.05,0.2]	Initial step sizes per gene
sigma_min	1e-5	Step size lower bound
tau	1/sqrt(6)	ES learning rate
seed	0	Random seed
attack_std	0.05	Noise attack standard deviation
snr_ref	40.0 dB	SNR normalisation reference for HoF score

Default settings: ~3200 evaluations (40 × 80). Runtime: 20–60 s.

12. Visualisation & Outputs

The standalone script saves watermark_results.png — a 3×4 matplotlib figure.

Row 0 — Signals

Original Audio	First 0.25 s of the clean signal
Watermarked Audio	Same window with HoF embedding
Difference	Embedded perturbation (watermark noise)
Bit Recovery	Original vs extracted bits, clean signal

Row 1 — Convergence

HoF Objectives	SNR and Bit Accuracy of HoF per generation (dual axis)
Front Size & Div.	Pareto front size + std(alpha); injection markers
HoF Chromosome	Normalised gene values [alpha, band, nc]

Row 2 — Analysis

Final Pareto Front	All individuals; rank-0 front highlighted; HoF marked
alpha Sweep	SNR and Acc vs alpha (band, nc fixed at HoF values)
band Sweep	SNR and Acc vs band (alpha, nc fixed at HoF values)

13. Streamlit Application

Hosted at markio.streamlit.app

13.1 Architecture

app.py uses a generator version of the GA that yields after every generation, allowing the progress bar and log to update in real time. Plotly replaces matplotlib for interactive charts.

```
def run_ga(audio, watermark, cfg):
    for gen in range(cfg['generations']):
        # ... one generation ...
        yield gen+1, hof, hist, note, None, None
    yield cfg['generations'], hof, hist, '__done__', pop, pareto
```

13.2 Sidebar Controls

Audio	Sample rate, duration, watermark bits
GA Parameters	Population size, generations, SBX eta, k_min/k_max, patience, injection fraction, seed

13.3 Live Run Log

Per-generation monospace log showing HoF's alpha, band, nc, SNR, Acc, and an injection badge when triggered.

13.4 Results Dashboard

Metric cards	SNR, Bit Accuracy, alpha, band, nc, Pareto front size with delta vs baseline
Gene bars	HoF chromosome values normalised to [0, 1]
Waveforms	Original, watermarked, difference
Audio players	Playback for original and watermarked
Convergence	HoF objectives + front size + diversity per generation
Pareto front	Interactive scatter with full population and rank-0 front
Gene sweeps	alpha sweep and band sweep at HoF's other-gene values
Bit recovery	Original vs extracted bits bar chart

14. Running the Project

Clone and install:

```
git clone https://github.com/futurevoid/MARKIO.git
cd MARKIO
pip install -r requirements.txt
# streamlit  numpy  plotly  matplotlib  scipy  soundfile
```

Standalone script:

```
python audio_watermarking_ga.py
# outputs: console log, watermark_results.png
```

Streamlit app (local):

```
streamlit run app.py
# http://localhost:8501
```

> Hosted web app: markio.streamlit.app

Extending:

Additional gene	Add row to Individual.BOUNDS and SIGMA0
Real audio input	Replace generate_audio() with soundfile.read() — drill.wav is in the repo
Blind detection	Remove original subtraction in extract_watermark()
Third objective	Add to compute_objectives() return; sorting and crowding generalise
Multiple runs	Loop main() over seeds; report mean ± std

15. Results & Analysis

Configuration	alpha	band	nc	SNR	Bit Accuracy
Baseline (fixed)	0.050	1.00	1	42.82 dB	87.5%
GA HoF	0.093	0.43	2	40.46 dB	98.4%
Delta	+0.043	-0.57	+1	-2.37 dB	+10.9%

Pareto front: ~10 non-dominated solutions. Diversity injections: 6.

band = 0.43. The optimiser moved away from full broadband toward a semi-narrow carrier, reducing high-frequency content in the watermark while maintaining robustness under the matched-filter detector.

n_carriers = 2. Two independent carriers per bit increase the correlation score at the detector (sum of two dot products vs one) at constant total embedding power, yielding disproportionate robustness gain.

alpha = 0.093 vs baseline 0.050. Diversity injections enabled the population to explore higher-alpha regions. The SNR cost (–2.37 dB) is acceptable relative to the 25 dB perceptibility threshold.

Pareto front. The front exposes the full SNR–robustness trade-off. Any point on it can be selected post-optimisation based on application requirements without re-running the GA.